CMPE 118

Professor Gabriel Hugh Elkaim

A brief introduction to Mechatronic Systems

William Cheng

Kenny He

Daniel Hunter

Fall 2016

CONTENTS

1 Hardware **2**
Introduction 2
Strategy 2
Mistakes and Lessons Learned 4
Results 6

2 Electronics **9**
Introduction 9
Strategy 9
Mistakes and Lessons Learned 11
Results 13

3 Software **16**
Introduction 16
Strategy 16
Mistakes and Lessons Learned 20
Results 21

4 Conclusion **22**

CHAPTER 1

HARDWARE

Introduction

Hardware was the most straightforward aspect of the robot. Several factors contributed to this. First, hardware at this scale is simply not that difficult, regardless of the resources at hand. With low velocity, low mass, and little to no impact forces, you can generally get away with a great deal of estimating. Secondly, the resources and training available to all teams were fantastic. Tab and slot construction along with a laser cutter and Solidworks made frame design simple. Easy access to foamcore and a test field allowed us to quickly prototype subsystems and test our assumptions.

Strategy

There were several main concerns we anticipated when designing the robot.

First, we wanted the robot to be easy to disassemble and repair, in preparation for the inevitable breakdowns. We wanted the circuits to be easy to access and remove, so we could fix any failures. Due to our short time frame and limited resources, we anticipated a significant amount of debugging in parallel with development. In order to reduce debugging time, we made all components as easy to access as possible. For this reason, we made the interior of the robot spacious and easy to access. The roof of our robot has no glue or bolts, and instead is held in place almost entirely by gravity (it can be attached more robustly if needed, but we have yet to find a situation in which that is needed). The side walls can also be quickly removed without the need for specialty tools, allowing access to our circuits and wiring.

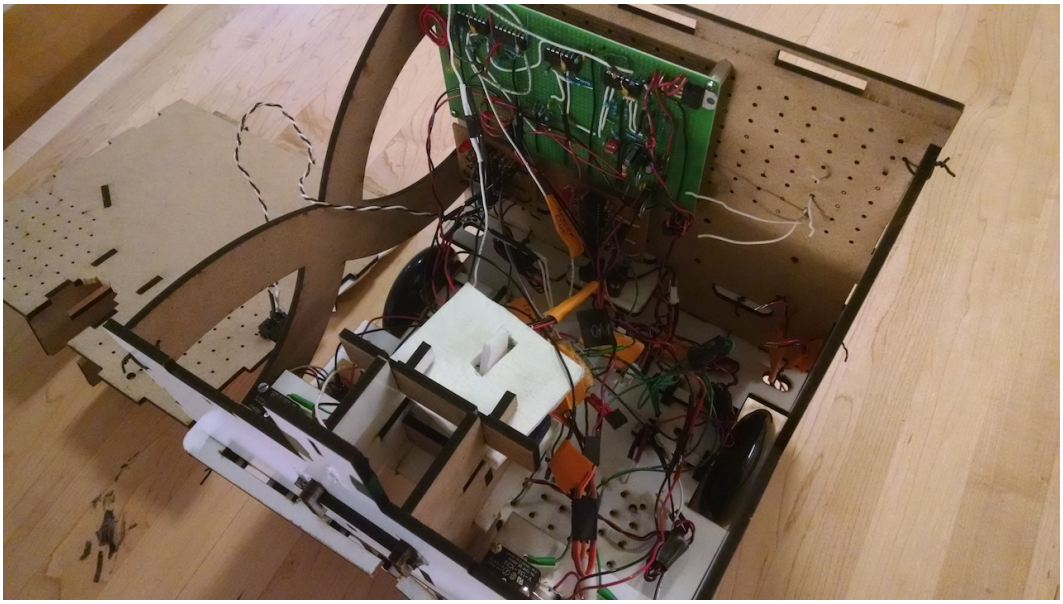

Figure 1.1: Spacious interior with easy access to circuitry

Second, we wanted an over sized funnel and plunger to reduce the need to align precisely with the ball loader. Although the track wire ended up making our alignments fairly exact, it was nice to never have to worry about lining up with the ball tower. Even if we were off angle, we would still effectively load balls. Of all the issues we encountered with the robot, we never had any issues loading ammo.

Third, we wanted all the components to be modular, such that one component could be modified or upgraded and still easily installed on the robot. We also wanted the ability to add entirely new components in the future, if issues arose that we didn't anticipate. This was accomplished by using a standard tab and slot attachment method wherever possible. As long as the tab and slot pattern was replicated on a new part, it could be swapped in easily. We also established a universal hole pattern on all of our parts such that we could easily mount new parts if needed. The hole pattern was also very useful for running wires throughout the robot. Instead of wondering where holes would be needed, we simply put them everywhere. This cost us time on the laser cutter, but saved us much more time when assembling and wiring the robot.

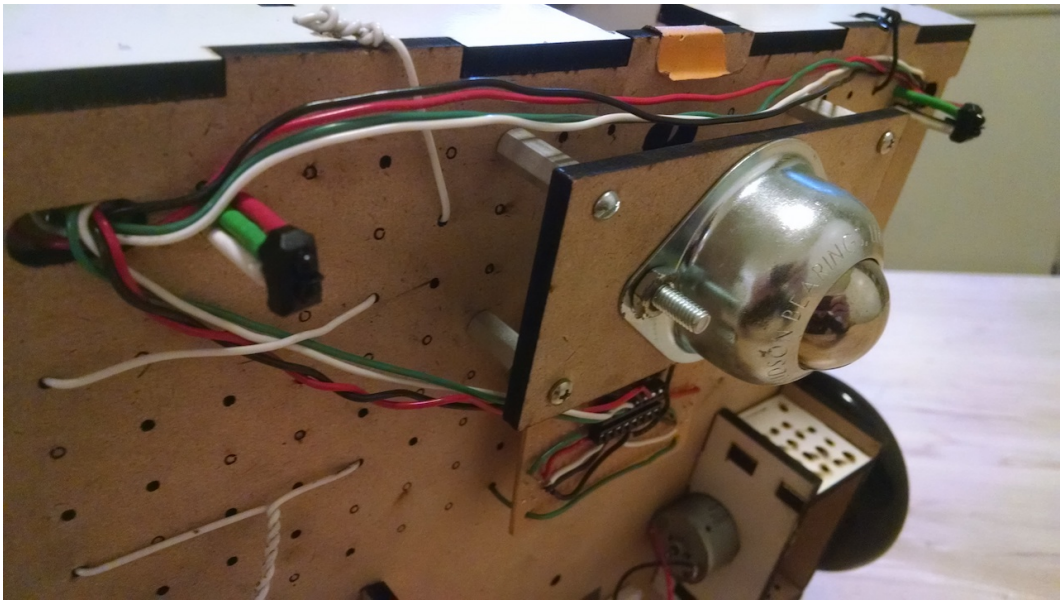

Figure 1.2: Universal hole pattern on the robot base being used to mount caster and run wires

Mistakes and Lessons Learned

While our large and boxy robot paid off when collecting ammo and making repairs, it ended up being undesirably top heavy. Too much acceleration would result in a wheelie, which would send garbage data to our tape sensors. In order to maintain integrity in our tape sensor data, we lowered the acceleration of our robot to avoid wheelies. This resulted in a slower overall robot, which was unfortunate. We made up for the slow acceleration with efficient software maneuvers, but we were still limited. If we were to design the robot over again, it would be with much less ground clearance to prevent wheelies, and a more skeletal frame — possibly using more foam core on top — to lower our center of mass.

Our ball tower plunger went through multiple designs as the weeks progressed. Initially, we were planning on a sponge or a brush. We bought a set of sponges from Safeway and tested those, which initially worked well. The main issues we encountered with sponges was the deterioration over time, and the difficulty mounting them to the robot. Sponges don't glue to things well, and one can't really use screws or bolts with them either. We managed to get by for a time using glue, but it was never a robust system.

We considered using a paint brush after seeing other teams have success with them. However, we also noticed a few teams having success with heatshrink. After an evening of rapid prototyping and gathering some empirical data, we decided to use a row of heatshrink tubes facing outwards at a slight downwards angle, mounted to the back of our robot. This allowed for an extremely flexible apparatus when hit from the side or top, while still remaining surprisingly stiff when force was applied from the back. We were able to rotate without worrying that the heatshrink was going to catch on something, but still be sure that

it would depress the plunger.

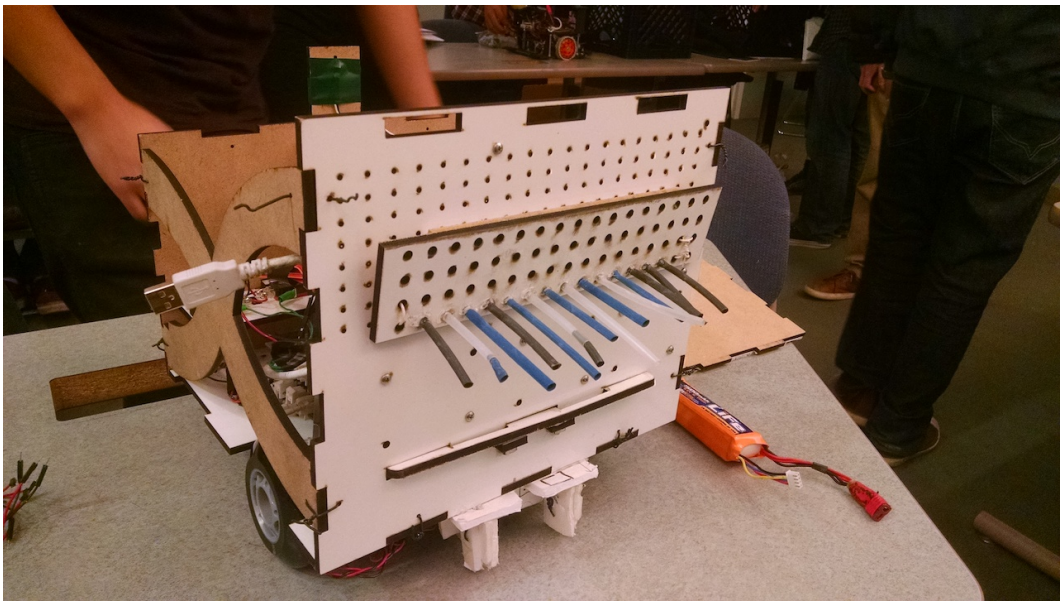

Figure 1.3: Heatshrink used to depress the ball plunger

Our roof/ball catcher also saw a few redesigns. Our initial funnel walls were too tall, which made our robot larger than the 11" x 11" x 11" size limit. This forced us to reduce the size of the walls preventing the balls from escaping the holding area. However, this made the ping pong balls more prone to bouncing out of the holding area, so we were forced to pad the surface with foamcore to reduce the bounciness.

Our initial gearbox positioning was another component that initially exceeded the size limit. Although the original CAD design in Solidworks told us that mounting the gearboxes $\frac{1}{2}$ inches inset from the edge would allow our robot to fit in the 11" x 11" x 11" allowance, the wheels ended up protruding out farther than we expected, forcing us to redesign the base plate with the gearboxes located even further in. Our modular tab and slot construction played a key role in making our new base plate easy to install. Instead of worrying about glue or bolts, we simply removed the top portion of our robot and added in the new base plate.

Another hardware issue we encountered was that our left motor would occasionally stall for a short period of time while being run, causing our robot to perform undesired turns. after close examination of the wheels, we noticed that this behavior was being caused by the wheel rubbing against one of the screws keeping the motor in place. Once we realized wheat was happening, we we able to easily resolve the issues by mounting the wheel slightly further away from the gearbox.

Results

The universal hole pattern was a definite success. Despite the time spent cutting it on the laser cutter, having it available in the future was extremely convenient and saved us a great deal of time and hassle. It came in handy a number of times, notably when running wires, mounting the battery, changing the location of our tape sensors, and when overhauling our ball tower plunger to use heatshrink.

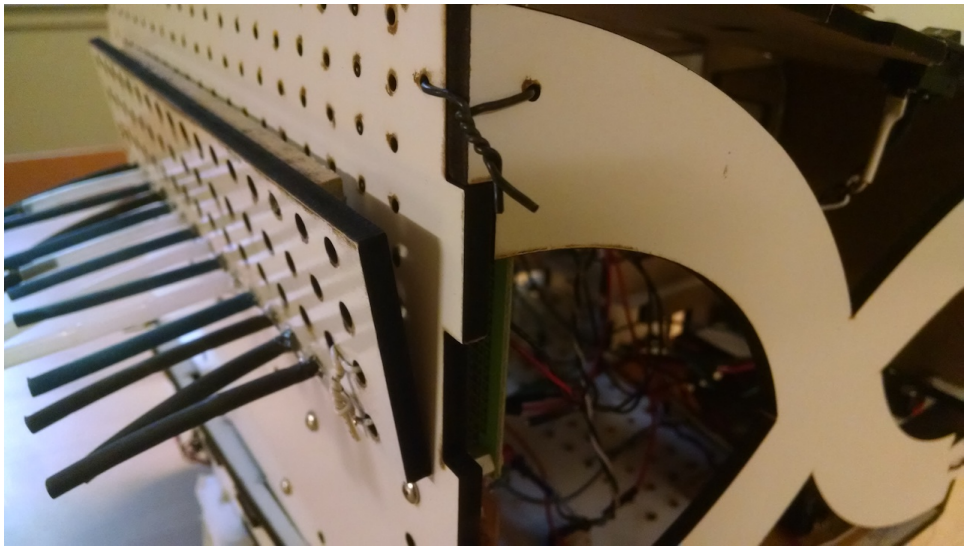

Figure 1.4: Universal hole pattern being used to mount heatshrink panel

Our modular design paid off as well — we were able to move very quickly once the drive base was finalized, before the rest of the robot was designed. The midterm was due on Tuesday, and by that Saturday (4 days later) we had a fully operational drive base that could track tape. Even though the top part of our robot was still in the design phase, we were able to cut the bottom plate, order motors, build tape sensors, and write the needed code. Working in parallel like this was crucial to our rapid development, and resulted in team 7 being the first team to complete an (unofficial) minimum specification run, and the second team to officially check off.

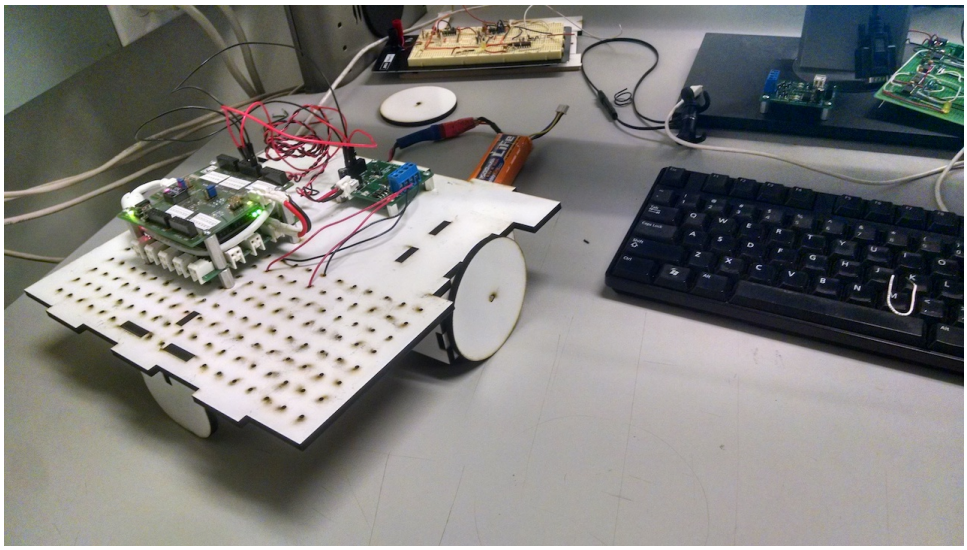

Figure 1.5: Initial robot base with MDF wheels, cut and assembled four days after starting the project

The funnel was successful at receiving balls from the tower and sending them in an ordered fashion to the gate release mechanism. Unfortunately, the release mechanism needed two full redesigns before it was sufficiently robust. Our initial design had a very weak servo mount, and used small foamcore panels to allow the balls to drop. The weak servo mount and undersized panels resulted in a weak system that broke easily. Our first redesign was more robust, with a much stronger servo mounting system and larger foamcore paneling. Unfortunately, our redesign had removed our two stage gate, which meant we had much less control over the ammo dropping. We had fixed some problems, but introduced others. Our second full redesign featured cotter pin construction (which lowered our reliance on glue), the same robust style of servo mount, and a two stage gate. After a bit of work cutting out various sized foamcore panels, we found a design that worked — every time. Initially we were worried about the longevity of the foamcore panels, and considered cutting MDF or acrylic paneling as a replacement. However, the foamcore proved to be reliable over time, never failing or deteriorating.

Our BOM is attached below:

	A	B	C	D	E	F	G	H	I	J
1	**Item Name**	**Quantity**	**Cost (Each)**	**Total Cost**	**Link**	**Notes**		**Total Cost:**	**68.14666667**	
2	MDF	3	$3.50	$10.50	N/A	900 sq in is a liberal estimate for one robot design. If we redesign significantly after cutting				
3	Motors	2	$12.99	$25.98	https://www.ama	Might need different motors, we'll see				
4	Limit switches	2	$0.71	$1.43	https://www.ama	Bought a pack of 15, will resell the remainder for a profit				
5	Servos	1	$3.00	$3.00		Obtained from Aaron				
6	Wheels	2	$4.00	$8.00		Obtained from Alec				
7	H Bridge	1	$0.00	$0.00		Provided by staff				
8	Uno Stack	1	$0.00	$0.00		Provided by staff				
9	Springs			$0.00		To push plunger. Might use sponge or something instead.				
10	Servo mounting screws	2	$0.00	$0.00		Comes with servos				
11	Motor mounting screws	4	$0.20	$0.80		M3 X .5 spaced .671 inches apart.				
12	Limit switch mounting screws	8	$0.09	$0.72		I have no idea why I bought 8, but that'll work just fine				
13	Ball Bearing Caster	1	$2.00	$2.00		Will obtain from Josh				
14	Circuit mounting screws	16	$0.17	$2.72						
15	Circuit mounting standoffs	10	$0.75	$7.50		2*(Track Wire + Track Wire + Beacon Detector + Tape Sensor + LED Diagnostic board)				
16	DIP socket	2	$2.00	$4.00		Bought from BELS, they charge so much :(				
17	Caster mounting standoff + nut	2	$0.75	$1.50		Didn't charge me for the nylock, yay				

Figure 1.6: Bill of Materials

CHAPTER 2

ELECTRONICS

Introduction

The electronics portion of our project consists of sensors, the hardware for the provided H-bridge and UNO, and the motors for our wheels. For the sensors, we used two track wire detectors, a beacon detector, four bumper switches, and three tape sensors. The only given hardware we used was the UNO stack and the H-bridge. Throughout the project, we made modifications and changed our decisions on some of the electronics implementation. Some examples include modifying our track wire circuit to read out analog values, replanning the number of tape sensors and their positioning, and adding more switches for the bumpers.

Strategy

The circuitry needed for our robot included two track wire sensors, a beacon detector, and three tape sensors. Our team already had a working beacon detector from lab 3, so we didn't need to worry about that. The tank sensors and the tape sensors needed to be designed and built from scratch. Thankfully, we could start immediately on the tank sensors, as we already had the schematic from lab 2. We were able to work in parallel on the circuitry, with some team members designing the robot and prototyping subsystems and other members working on circuitry. This allowed us to move very quickly once we had a driving base, and begin integrating sensors into the code. This style of simultaneous and modular development was a cornerstone to our project, and allowed us to develop our robot (and more importantly, identify mistakes) very quickly.

We decided to implement two track wire detectors on our robot — one in the front and one in the rear. We first tested the track wire detector circuit design on a proto-board. Once every component was tested and working, we designed a circuit schematic for a perf-board. Then, we soldered up two complete track wire circuits on two seperate perf boards.

The inductor was attached to the circuit using male headers and detachable female jumpers. This allowed the inductor to be easily detached if needed, allowing for easy repairs or modifications. At one point we needed to lengthen the attachment wire, as we were changing where the inductor was mounted. If we had soldered the inductor directly to the board, this would have been a hassle to do. As it was easily removable, we simply pulled it out, soldered a longer wire on, and plugged it back in. This allowed one team member to work on the inductor, while the other team members continued to test the robot and work on code. Working in parallel like this allowed us to develop very quickly, and test critical subsystems early.

The tape sensor schematic is shown below

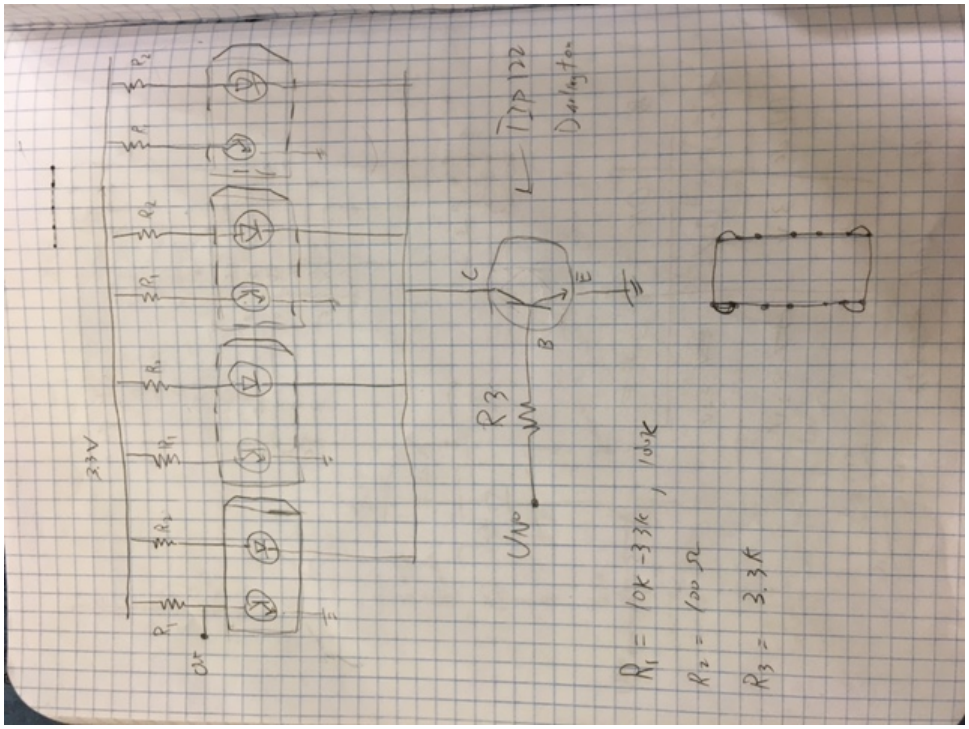

Figure 2.1: Tape sensor circuit schematic

For the tape sensors, we began designing the circuit schematic with five tape sensors and synchronous sampling in mind. We had five tape sensors in parallel in the sourcing configuration. To implement synchronous sampling, we needed to have a way to turn the tape sensors' LEDs on and off. To do so, we had the low side of the LEDs go to a collector of a TIP122 transistor. The emitter went to ground. We wired a digital output from the UNO32 to the base of the transistor. The transistor would act like a switch that lets the low side of the tape sensors' LED go to ground or open circuit. The switch would be controlled by our digital output at the base of the transistor. We tested the schematic on a bread board before we soldered the components onto a perf board. The tape sensor circuit schematic is shown above.

First, we split the schematic into two perf boards. The first board takes care of all the circuit logic such as the transistor, resistors, and power regulator. The second board was strictly used to hold the tape sensors. The tape sensors' leads were all soldered to a dip socket. The circuit logic board would also have a corresponding dip socket. Because the tape sensor board would be held on the bottom of our robot, the dip sockets allowed us to conveniently connect and run wires to the circuit logic board which was inside our robot.

Initially we planned to have two bumper switches in the front — left and right — and one bumper on the back. The limit switches each had three terminals with one main terminal that is connected at all times to either of the two other terminals. When the switch was press, the main terminal disconnected from the first terminal and connected to the second terminal. The main terminal was our input into the UNO. The first terminal was grounded and the second terminal was connected to a 3.3V rail. Our input to the UNO switched between 0V and 3.3V, giving a high or low reading based on the state of the bumpers.

After testing this system, we determined that the back bumper was not needed, and that another pair of limit switches was needed in the front to fully detect bump events. After adding in two more switches — for a total of four — in the front, we were able to detect bump events while driving forward or turning.

For the motors, we used the H-bridge that was given to us. The H-bridge had the capacity to power two motors which was all we needed. We had two PWM outputs and two digital outputs from our UNO going to the H-bridge. The PWM controlled the motor speed of the motors and the digital output controlled the direction of the motors.

Mistakes and Lessons Learned

In the lab manual, a comparator circuit is used on both the beacon detector and the tank circuits. For the final project, using the comparator turned out to be extra work and resulted in a reduction in functionality. While having a clean binary value is essential for using the I/O ports on the Uno, there are enough ADC ports provided that it makes more sense to use those instead of the standard digital I/O ports. Using the ADC ports provides data with more resolution, which allows the software to be more intelligent. Rather than send an event whenever we received a logical high out of the comparator chip, we created algorithms that analyzed each reading from the peak detector, and determined precisely when the signal was at its maximum. This allowed us to be much more precise with our track wire detection, which in turn reduced problems loading ammo.

If we had instead taken the output of the comparator chip, we would have seen a logical high as we approached the track wire, and then a logical low some unknown time later as we drove away from the track wire. Due to varying power from track wire to track wire, this presented a problem of accuracy. This problem can be dealt with by using a timer to find the mid point between the two events, but we feel that our method of track wire detection is cleaner and more reliable.

The event checkers we used are discussed in more detail in the software section of this document, but we have included the peak detection algorithm here.

```
currentBackTrackWire = AD_ReadADPin(BACK_TRACK_WIRE_SENSOR_PIN);

if (currentBackTrackWire > BACK_TRACK_WIRE_DATA_THRESHOLD) {
    if (currentBackTrackWire > previousBackTrackWire) {
        previousBackTrackWire = currentBackTrackWire;
        backTrackWireDebounceVar = 0;
    } else {
        backTrackWireDebounceVar++;
    }
} else {
    previousBackTrackWire = 0;
}

if (backTrackWireDebounceVar > BACK_TRACK_WIRE_DEBOUNCE_THRESHOLD) {
    curEvent = BACK_TRACK_WIRE_DETECTED;
} else {
    curEvent = BACK_TRACK_WIRE_LOW;
}
```

Once the current track wire reading is less than the previous track wire reading (which occurs once the track wire has been passed), the debounce variable begins incrementing. If the current reading remains lower than the previous reading for a sufficient amount of readings (which serves to debounce the readings), then an event is triggered. This serves to find the peak power from a track wire, and trigger an event a split second after the robot passed the peak. We experimentally verified the debounce threshold to ensure that it was not excessively large, as that would cause the robot to overshoot the trackwire.

A lower level mistake was forgetting to put dipsockets on the perfboard to allow for easy chip swapping. If a chip was blown on any of our perfboards, we would have been forced to desolder all 14 pins on that chip to extract it. Although this mistake did not prevent us from having working track wire detectors, we did have to divert additional time to adding dipsockets to the perfboards.

Some problems we had with the bumper switches included broken switches and unstable event readings. For broken switches, we simply swapped them out. We found out later that the unstable event readings were caused by loose connections. We fixed this problem by soldering the wires directly onto the switch terminals. Our initial design had 3.3V running directly into the limit switch, and then into the UNO. When the switch was pressed, our 3.3V power rail was essentially shorted directly into the UNO. This was a mistake, and cost us several days debugging before we realized that as current limiting resistor was the solution. Without the resistor, the ES˙Framework would crash constantly (as the UNO was restarting).

Results

The dual track wire sensor set-up was a resounding success when combined with our advanced software peak detector and alignment algorithm. The inductor at the back of the robot was used to fix our translational position, and the front inductor was used to align our rotational position. After measuring the strength of the track wire signal and tuning the wheels' drive ratios, we were able to successfully load balls from all of the towers.

The schematic of our track wire detector is shown below, in figure 2.2. The represents the final state of our track wire detector circuit, after all of the changes we made during the development process.

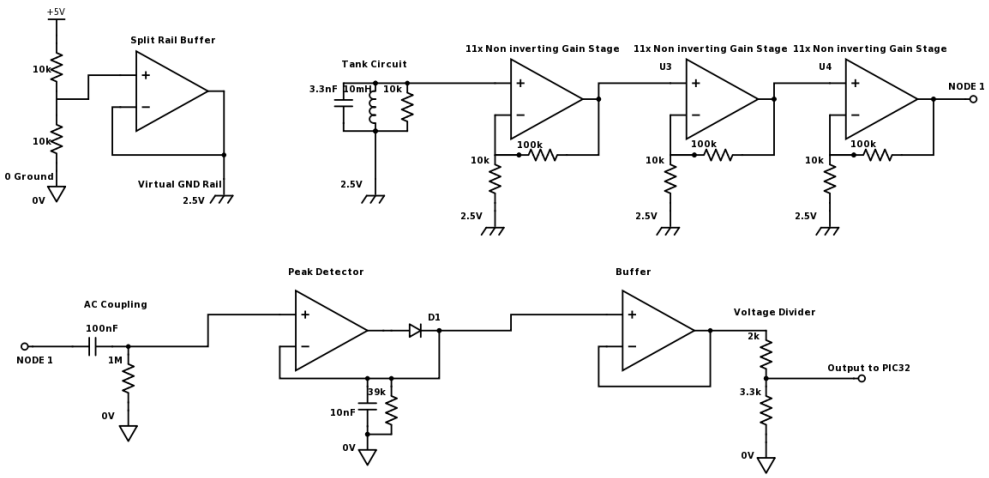

Figure 2.2: Schematic for our TrackWire Detectors

The layout of components on the perfboards for each of our Track Wire Detectors is shown below in figure 2.5. Be aware that the layout includes vestigal modules that went unused in the final design, such as the post-peak detector gain stage and comparator.

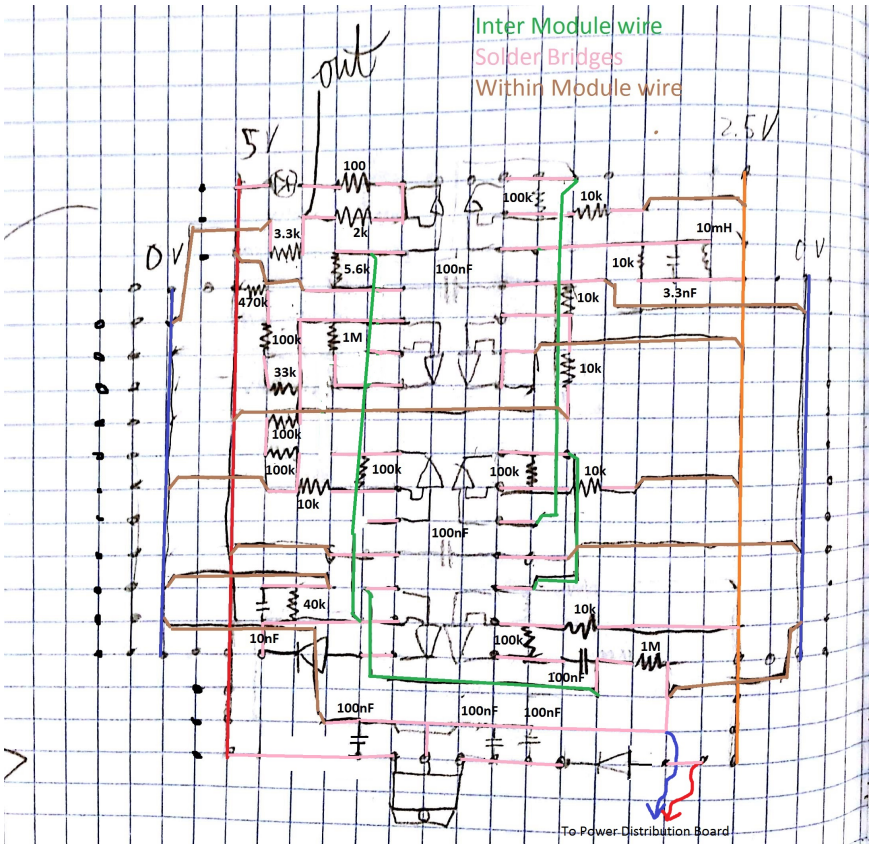

Figure 2.3: Layout for our TrackWire Detectors

We soldered up the tape sensor schematic with room for five sensors. The dip sockets allowed us to conveniently add and remove tape sensors throughout the project. We first started with three tape sensors in a triangle formation near the front of the robot. Later, we moved the left and right tape sensors to the front corners of our robot. The dip sockets allowed us to easily rewire the tape sensors and allowed us to add two more tape sensors if we needed to. In the end, we stayed with one tape sensor near the front middle and a sensor on each front corner.

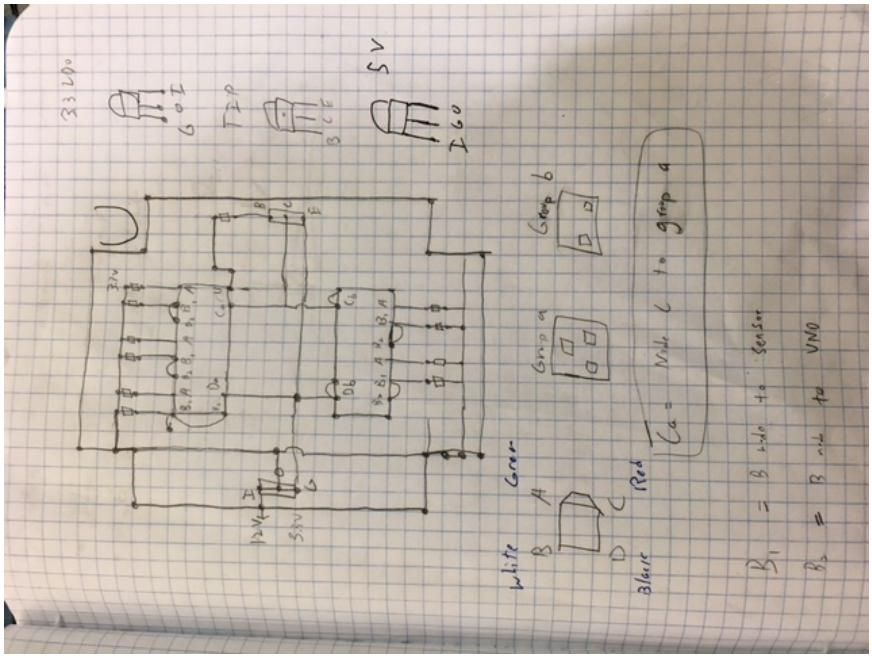

Figure 2.4: Tape Sensor Logic Perf Board

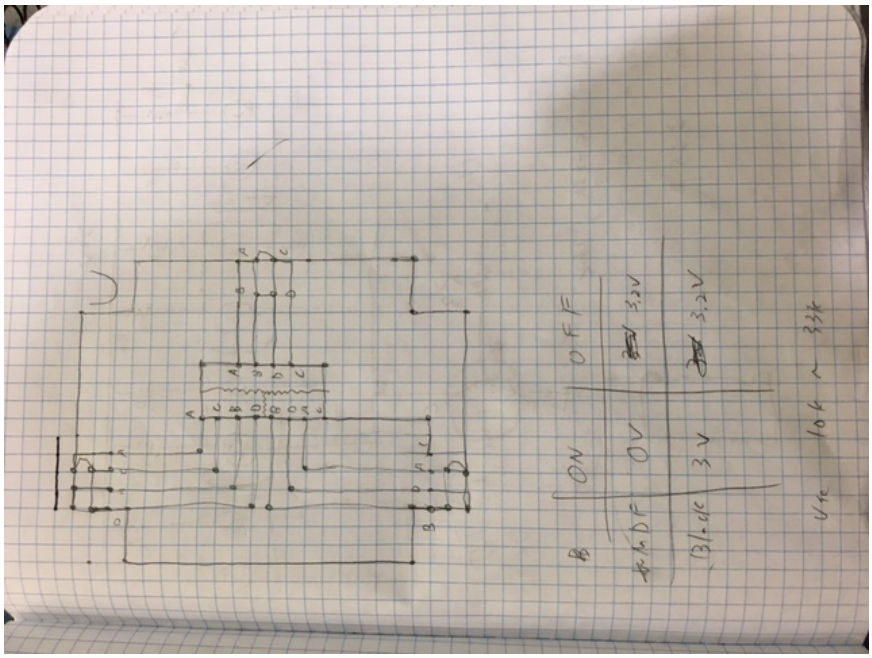

Figure 2.5: Tape Sensor Perf Board

CHAPTER 3

SOFTWARE

Introduction

Our code base consisted of a service to implement synchronous sampling, various event checkers, and our main HSM. We used a private GitHub repo to maintain a backup of our codebase. Using GitHub proved to be very useful, as we repeatedly had to roll back changes to fix errors. We saved a great deal of time by checking the differences between commits and finding errors as opposed to manually hunting through code. GitHub was also very useful for storing our Solidworks files. Throughout the initial design phase of the project, significant changes were made that we occasionally wanted to revert. Having the ability to easily download the robot design at specific points in time was useful on multiple occasions, and saved us a great deal of time and frustration. This also allowed us to transition between computers or simultaneously write code. At one point the team decided to transition from room 115 to room 113. Using the 'git clone' command allowed us to quickly download our entire codebase and continue work without any issues.

Strategy

We wanted to create software that was as robust and easy to debug as possible. To that end, helper functions and pound defines were utilized as much as possible. Following are some code snippets to demonstrate this.

```
// Motors
#define MAX_MOTOR_SPEED 900
#define MEDIUM_MOTOR_SPEED 600
#define SLOW_MOTOR_SPEED 450

#define FORWARD 0
```

```
#define REVERSE 1

#define LEFT_MOTOR_PWM_PIN PWM_PORTY12
#define RIGHT_MOTOR_PWM_PIN PWM_PORTY10

// Tape
#define BLACK_TAPE_THRESHOLD 450
#define WHITE_THRESHOLD 300

// Left and Right is from robot POV
#define LEFT_TAPE_SENSOR_DATA_PIN AD_PORTV4
#define CENTER_TAPE_SENSOR_DATA_PIN AD_PORTV6
#define RIGHT_TAPE_SENSOR_DATA_PIN AD_PORTV8

#define LEFT_TAPE_SENSOR_MASK 0x01
#define CENTER_TAPE_SENSOR_MASK 0x02
#define RIGHT_TAPE_SENSOR_MASK 0x04

// Beacon Detector
#define BEACON_DETECTOR_PIN AD_PORTV3
#define BEACON_DETECTOR_THRESHOLD 550
#define BEACON_DEBOUNCE_THRESHOLD 200

// Bumpers
#define FRONT_RIGHT_OUTER_LIMIT_SWITCH_PIN PORTX09_BIT
#define FRONT_RIGHT_INNER_LIMIT_SWITCH_PIN PORTX10_BIT

#define FRONT_LEFT_INNER_LIMIT_SWITCH_PIN PORTZ05_BIT
#define FRONT_LEFT_OUTER_LIMIT_SWITCH_PIN PORTZ06_BIT

// Track Wire
#define BACK_TRACK_WIRE_SENSOR_PIN AD_PORTV7
#define FRONT_TRACK_WIRE_SENSOR_PIN AD_PORTV5
#define BACK_TRACK_WIRE_DATA_THRESHOLD 450
#define FRONT_TRACK_WIRE_DATA_THRESHOLD 450
#define BACK_TRACK_WIRE_DEBOUNCE_THRESHOLD 10
#define FRONT_TRACK_WIRE_DEBOUNCE_THRESHOLD 10

// Ball Drop Servo
#define BALL_DROP_SERVO_PIN RC_PORTZ08
#define SERVO_POSITION_ONE 1500
#define SERVO_POSITION_TWO 2000
```

This snippet is from the beginning of our helper function .h file. Maintaining a master list of thresholds, ports, masks, and other useful data was invaluable to our ability to write new software modules quickly and without error. This list was maintained throughout the project, referenced consistently when debugging hardware, and referenced in new MPLAB X projects as needed. Instead of creating duplicates of the file (and then inevitably wondering which version we were working with for any given project), we created one master file and

referenced it in all of our state machine and test harnesses. Once we established a function was operational using our test harness, we could immediately use that exact same function in our full state machine without worrying that it wasn't working properly.

Our helper functions themselves saved us a great deal of time. The time spent to write effective helper functions paid off hundreds of times over throughout the project. We built each function and tested them in isolation using our test hardness. Once we were satisfied with the functionality, we implemented them into the state machine. Heavy use of well named helper functions kept our code short, readable, and easy to debug. Following are the headers for some of our more commonly used functions.

```
/**
 * @Function motorsOff
 * @param None
 * @return None
 * @brief Helper function used to turn all drive motors off
 * @author Daniel Hunter, 6 Nov 2016 */
void motorsOff();

/**
 * @Function driveForward
 * @param dutyCycle - the desired PWM duty cycle
 * @return None
 * @brief Helper function used to drive the robot forward at various speeds
 * @author Daniel Hunter, 5 Nov 2016 */
void driveForward(int dutyCycle);

/**
 * @Function driveBackward
 * @param dutyCycle - the desired PWM duty cycle
 * @return None
 * @brief Helper function used to drive the robot backward at various speeds
 * @author Daniel Hunter, 5 Nov 2016 */
void driveBackward(int dutyCycle);

/**
 * @Function leftMotor
 * @param direction - the direction of the left motor
 * dutyCycle - the desired PWM duty cycle of the left wheel
 * @return None
 * @brief Turns the left motor as desired, and SHUTS THE RIGHT MOTOR OFF
 * @author Daniel Hunter, 11 Nov 2016 */
void leftMotor(int direction, int dutyCycle);

/**
 * @Function rightMotor
 * @param direction - the direction of the right motor
 * dutyCycle - the desired PWM duty cycle of the right wheel
 * @return None
 * @brief Turns the right motor as desired, and SHUTS THE LEFT MOTOR OFF
```

```
 * @author Daniel Hunter, 11 Nov 2016 */
void rightMotor(int direction, int dutyCycle);

/**
 * @Function toggleServo
 * @param None
 * @return None
 * @brief Toggles the servo gate between positions. Calling this function twice
 * with ~500 ms between calls should release a ball
 * @author Daniel Hunter, 15 Nov 2016 */
void toggleServo();

/**
 * @Function CustomPWM_SetDutyCycle
 * @param dutyCycle - the duty cycle to be set to the appropriate motor
 * @return None
 * @brief Uses the voltage reading from the battery and modifies the duty cycle
 * such that the motor speed does not flucuate with battery levels
 * @author Daniel Hunter, 19 Nov 2016 */
char CustomPWM_SetDutyCycle(unsigned char Channel, unsigned int Duty);

/**
 * @Function variablePercentLeftTurn
 * @param dutyCycle - the desired PWM duty cycle of the full speed wheel
 * precentage - the ratio between the full speed wheel and the slower wheel
 * @return None
 * @brief Helper function used turn the robot gradually
 * @author Daniel Hunter, 5 Nov 2016 */
void variablePercentLeftTurn(int percentage, int dutyCycle);

/**
 * @Function variablePercentRightTurn
 * @param dutyCycle - the desired PWM duty cycle of the full speed wheel
 * precentage - the ratio between the full speed wheel and the slower wheel
 * @return None
 * @brief Helper function used turn the robot gradually
 * @author Daniel Hunter, 5 Nov 2016 */
void variablePercentRightTurn(int percentage, int dutyCycle);
```

The dates on the functions are worth noting — due to our modular pound defines, we were able to write the functions even before the robot design was finalized. Some of the functions were written only a few days into the project, despite not having full robot designed yet. Once we determined our high level strategy we were able to identify specific needs for helper functions, and create them. The pound defines could be easily changed later according to the hardware.

A great deal of our software was fairly standard code that most teams had to implement. Functions to drive forward, turn, activate a servo — none of that is incredibly advanced software. However, we did write some code that we believe is worthy of mention.

One such function is our CustomPWMSetDutyCycle function. Using battery voltage readings and some calculations, we were able to ensure that the motors always received the same amount of voltage, even as our battery drained. Due to our reliance on timers, this was a key function to maintain accuracy and reliability over many runs. We were able to execute turns with precise accuracy (+/- 5 degrees on a full 360 degree turn) and drive forward or backward precise distances when needed.

```
char CustomPWM_SetDutyCycle(unsigned char Channel, unsigned int Duty) {
    unsigned int newDuty = 0;
    unsigned int batVoltage = 0;
    double newBatVoltage = 0;
    batVoltage = AD_ReadADPin(BAT_VOLTAGE); // read the battery voltage
    newBatVoltage = (double) batVoltage;
    if (newBatVoltage != 0) {
        newBatVoltage = newBatVoltage / 1023 * 33; // = current actual battery voltage
        newBatVoltage = newBatVoltage - 0.6; // account for diode loss
        newDuty = Duty / newBatVoltage * 9.9; // Modify the duty cycle as needed to
        // account for a lower bat voltage
    }
    if (newDuty > 1000) {
        newDuty = 1000; // if the duty cycle has exceeded the limits, trim it
    }
    return (PWM_SetDutyCycle(Channel, newDuty));
```

Mistakes and Lessons Learned

One of the more significant obstacles we encountered was that of event spam. The weekend before beer checkoff, we spent over 48 hours doing nothing but debugging our event checkers. The terminal would spam two different events, one after the other, continuously. Which two events it spammed seemed to change as we debugged the error, making debugging even more difficult. Even worse, the two events weren't connected. If we had seen 'BUMPER_HIGH' and 'BUMPER_LOW' events spammed (one after the other), that's not that hard to debug and test. In that scenario, maybe debouncing is needed, or perhaps there is a bad connection on the bumper switch. The event spam we experienced was between events like 'BEACON_DETECTED' and 'BATTERY_CONNECTED'. Another combination we saw was 'BATTERY_CONNECTED' and 'ES_NO_EVENT'. Due to the seemingly random pairing of the events that were being spammed, debugging was a confusing process. Over the several days (and nights) we spent debugging the issues, we made several changes that we believe contributed to solving the problem.

First, we changed all event checkers to only post a binary result — that is, it only posted one of two events. Originally, we were experimenting with event checkers that posted one of several events (LEFT_BUMPER_HIGH vs RIGHT_BUMPER_HIGH vs NO_BUMPERS), but we opted to instead have dedicated event checkers for different systems (CheckLeft-

Bumpers and CheckLeftBumpers). This allowed us to further isolate the different events, and continue debugging. In the process of modifying our event checkers, we also removed all of them from the ES_Configure.h file, and then incrementally added them back in, one at a time. This allowed us to further isolate which functions were spamming events.

Second, we added a current limiting resistor to the power rail that was running to the limit switches. Due to the amount of current being sunk into the UNO, we were worried that this was causing undefined behavior and contributing to the event spam. This was a turning point in the debugging — up until that point, we had been trying to fix the issue using software. We had made the classic mistake of going straight to software without first ensuring the integrity of the hardware. Once the resistor was added, we immediately saw much more reliable behavior and were able to debug more efficiently.

Third, we replaced one of the limit switches. We determined that one of our limit switches was broken — we believe due to the impact forces of driving into the ball tower repeatedly. The broken switch was oscillating between closed and open as the bumper wiggled back and forth against the switch. From what we were able to determine, it seemed that the switch was still operating as a switch, it just had an extremely weak actuation force, and was being triggered by the slightest of vibrations. This was contributing to the event spam, and also sinking a great deal of current into the UNO even when it did not appear that the bumper was being pressed.

Once we had made these three changes, the system was functioning normally again, and we were able to continue our progress.

Another issue was that of state machine depth. As we built our min spec state machine, we only went two levels deep — we didn't encounter scenarios where we needed more depth. Once we achieved min spec and transitioned to refinement and ensuring robustness for corner cases, we found that our depth was inadequate. Adding in more levels was occasionally possible, but often we were limited by our previous code in how much extra intelligence we could add. Had we understood how beneficial it is to have a three or four level state machine designed from the start, we would have had much more ability to 'future proof' our code. By taking a bit of a shortcut in the beginning, we paid for it down the line as it became difficult to add extra functionality.

Results

Despite our setbacks, our state machine ended up being quite robust and able to handle various corner cases. As the second team in the class to get checked off, we owe much of our sucess to our parallel development strategy, which allowed us to begin testing and navigating corner cases earlier.

For the beer challenge, we built a field mapping system that determines how many beacons are in range of the robot, and use that information to find its way to the edge of the field.

CHAPTER 4

CONCLUSION

Although we were not able to pass the beer challenge or properly compete in the competition, this project allowed all team members to gain more experience working with hardware, electronics, and software by developing our robot. Despite the enormous investment of time, this class was very enjoyable and the entire time had a great experience. All of us on team 7 would like to thank Professor Elkaim, Max, and the staff of TAs and tutors for all their hard work. The class was fantastic only due to their help.

The team has gained enormous perspective and context for the mechatronics industry, and had learned a great deal from the fantastic 10 week experience. Team 7 feels proud to have made it through CMPE 118, and we are all happy with the robot we built.